

Early warning system at pedestrian crossings

Documentation

1. Introduction

1.1. Project overview

The objective of this project is to develop an early warning system at pedestrian crossings that alerts pedestrians to dangerous road situations. The system triggers an alarm 5 to 15 seconds before a vehicle reaches the crossing. Detected hazardous events include speeding, overtaking, and lane departures.

1.2. Project objectives

- Threat detection and alarm activation 5 to 15 seconds before a vehicle reaches the crossing zone.
 - Monitoring and classification of three types of dangerous events: speeding, lane departure, and dangerous overtaking within the approach zone up to 15 meters before the crossing.
 - Road characteristics: Straight, one-way, two-lane section with a minimum visibility of 200 meters and clear horizontal road markings.
 - Environment: Operation in diverse weather conditions (including rain, fog, and black ice) and a clear field of view for sensors without objects obstructing the frame.
 - Sensor structure: Data fusion from a video stream of a single camera with a fixed field of view and a sample point cloud from a radar.
 - True Alarm Rate (TAR) - above 80%
 - False Alarm Rate (FAR) - below 20.00%
-

2. Project Structure

2.1. General Project Structure

```
ZSD2026-SW0/  
├── CARLA_dev/  
├── data/  
├── Detection_dev/  
│   ├── algorithms/  
│   │   ├── car_detection/  
│   │   ├── lane_detection/  
│   │   └── model_training/  
│   ├── utils/  
│   ├── batch_runner.py  
│   ├── main.py  
│   ├── main_hardware.py  
│   └── setup.py  
├── docs/  
├── hardware/  
├── Scenarios/  
├── sensors/  
├── README.md  
└── README_PL.md
```

2.2. Description of Selected Directories

2.2.1. Data Directory

```
data/  
├── dataset/  
│   ├── alarm/  
│   └── noalarm/  
├── config/  
│   └── config.json  
└── models/  
    ├── best.pt  
    ├── model_quant_edgetpu.tflite  
    └── depth_anything_v2_vits.pth
```

This directory contains a complete set of dangerous and control scenarios: normal traffic, several variants of overtaking, drifting onto the roadside, and speeding (details in [Section 6.2](#)), a configuration file generated by the initialization script ([Section 6.1](#)) and the models used: Depth-Anything-V2 (depth estimation) and YOLOv8 adapted (via transfer learning) to the CARLA simulation environment.

2.2.2. Detection Module

```
Detection_dev/
├── algorithms/
│   ├── car_detection/
│   ├── lane_detection/
│   │   └── lane_detector.py
│   └── model_training/
├── utils/
│   ├── alarm_manager.py
│   ├── car.py
│   ├── depth_v2.py
│   ├── radar.py
│   ├── utils.py
│   └── weather.py
├── batch_runner.py
├── main.py
├── main_hardware.py
└── setup.py
```

The main logic of the project is concentrated in this directory:

- **main.py** – the main program of the project.
- **main_hardware.py** – a variant of the main program optimized for execution and performance benchmarking on the physical testing platform.
- **batch_runner.py** – a script for automated system testing by generating output metrics (discussion of metrics in [Section 9](#)).
- **setup.py** – an initialization script that configures the environment and dependencies of the detection module ([Section 6.1](#)).

The utils subdirectory:

- **utils.py** – this script implements the overtaking detection logic ([Section 5.2](#)), as well as auxiliary functions: visualization of Bounding Boxes for vehicles detected by YOLO, generation of comparison plots, and saving object telemetry data to .csv files.
- **car.py** – the Car class and data structures representing the dynamic state of a detected vehicle (position trajectory, velocity vector, physical dimensions, tracking history). This class is responsible for radar data fusion, estimating object dimensions in meters, and detecting uncontrolled lane departures ([Section 5.3](#)).
- **radar.py** – a module for analyzing signals from the radar sensor, performing filtering, clustering, and geometric detection of objects in 2D space ([Section 5](#)).
- **weather.py** – integration with an external weather API. Fetching relevant data for a selected location and determining weather-dependent parameters ([Section 5.1.3](#)).
- **depth_v2.py** – the module responsible for depth estimation. ([Section 5](#))

The algorithms subdirectory:

- **lane_detection (lane_detector.py)** – algorithms responsible for the segmentation, extraction, and tracking of lines defining traffic lanes on the roadway.
- **car_detection** – initial versions of the YOLO implementation.
- **model_training** – training scripts: the process of transfer learning for the YOLOv8 model on CARLA simulation data, and an archival pipeline for generating data for a CNN classifier evaluating vehicle types.

2.2.3. Other Directories and Files

- **CARLA_dev** - Files for data generation in the CARLA simulation environment.
- **docs** - documentation.
- **hardware** - benchmarks, metrics, and configuration files related to the testing platform.
- **Scenarios, sensors** - first versions of scenario implementations and sensor configurations, preserved as archival versions.
- General information regarding the project in Polish and English can be found in the **README** files.

3. Licenses and Copyrights

The project implementation utilizes the integration of open-source components, with all used tools and datasets applied in accordance with their applicable license terms.

Tool	License
Python (>3.8.6)	PSF License Version 2
OpenCV (>4.5.0)	Apache License 2.0
CARLA (0.9.16)	MIT License
Ultralytics YOLO (YOLOv8)	GNU Affero General Public License v3.0
Depth-Anything-V2	Apache License 2.0
open-meteo	Creative Commons Attribution 4.0 International (CC BY 4.0) licence

Table 1 - Software licenses

4. Technology Stack

The project utilizes the following technological solutions to create the Early Warning System:

4.1. Software:

- Python (>3.8.6) – the primary programming language used to implement detection algorithms and system logic.
- OpenCV (>4.5.0) – used for real-time image processing.
- CARLA (0.9.16) – a simulator used for data generation, road environment configuration, and scenario testing.
- Ultralytics YOLO (YOLOv8) – a detection model responsible for continuous vehicle identification and trajectory tracking.
- Depth-Anything-V2 - a model responsible for determining image depth to estimate the positioning of detected vehicles.
- Open-Meteo - a weather API used to determine current road conditions at a selected location.

4.2. Hardware:

- Raspberry Pi 4 B – test platform.
- Google Coral Edge TPU – hardware accelerator .

4.3. Used Libraries:

- **Ultralytics YOLO**: performs vehicle class object detection and provides dynamic bounding boxes for lateral and longitudinal position analysis.
- **torch, torchvision** – execution environment for deep learning models.
- **NumPy**: performs high-performance matrix operations, 3D coordinate transformations (radar rotation correction), and depth map transformations.
- **scikit-learn** (sklearn.cluster.DBSCAN module) – point cloud segmentation algorithm.
- **OpenCV (cv2)**: handles video streams, graphic mask operations, lane line approximation, and generating Graphical User Interface (GUI) overlays for Level 1 and Level 2 alerts.
- **pandas** – fast loading, filtering, and analysis of radar data from CSV files.

- **psutil** – monitoring and sampling CPU utilization (cpu_percent) for optimization purposes.
 - **matplotlib** – generating diagnostic plots and 3D scene visualizations.
 - **requests** – handling HTTP requests to the weather station REST API.
-

5. Operation of the Detection System

5.1. Speeding (Code A)

5.1.1. Assumptions

The system identifies vehicles moving at a speed unsuited to regulations or prevailing road and atmospheric conditions by dynamically calculating an extended braking distance.

5.1.2. Methods of Determining Vehicle Speed

Speed is retrieved directly from a physical radar synchronized with the video stream. At each time step, the point cloud is spatially filtered (masking longitudinal distance Y and height Z). To extract individual vehicles, raw points are vector-scaled using weights and subjected to segmentation via the DBSCAN algorithm. A stable mean radial velocity and center of mass are calculated for each clustered object. To eliminate mounting errors, the system automatically computes the yaw and roll angles of the point cloud and corrects them in 3D space using a rotation matrix. Furthermore, the constant offset between the radar and YOLO is linearly approximated by a regression function with a Z-Score anomaly filter.

5.1.3. Criteria for Raising an Alarm

Level 1 Alarm (Warning): Triggered when a vehicle's speed (the mean radial velocity of the cluster, marked negatively due to the approach vector) exceeds a fixed threshold value of 60 km/h.

Level 2 Alarm (Critical): Triggered when the estimated braking distance of the vehicle does not allow it to stop before the crossing. The system fetches weather conditions via the Open-Meteo API to determine a road adhesion multiplier. Driver reaction time is a non-linear variable dependent on the weather. The dynamic safe stopping distance (d_{stop}) for velocity v (in m/s) is calculated as:

$$d_{stop} = (t_{reaction} \cdot v) + [3 \cdot (\frac{v}{10}) + (\frac{v}{10})^2] \cdot multiplier$$

If d_{stop} is greater than the distance to the pedestrian crossing, the system raises a level two alarm.

5.2. Driving Anomaly (Code B)

5.2.1. Assumptions

The system monitors the stability of the driving direction to detect events such as uncontrolled drifting out of the lane.

5.2.2. Traffic Lane Analysis

The lines bounding the traffic lanes are defined in the system configuration using linear function parameters calculated relative to the vertical axis of the image Y (due to camera perspective):

$$x = m \cdot y + b$$

For each frame, the bounding box coordinates of the vehicle (x_1, y_1, x_2, y_2) are retrieved. The system then calculates the theoretical, safe boundaries of the left and right lines for the current position of the vehicle's lower edge (y_2) :

$$\begin{aligned} x_{left\ line} &= m_{left} \cdot y_2 + b_{left} \\ x_{right\ line} &= m_{right} \cdot y_2 + b_{right} \end{aligned}$$

5.2.3. Criteria for Raising an Alarm

A lane anomaly alarm is generated if the extreme points of the vehicle's bounding box cross the designated lane boundaries, accounting for an error tolerance equal to 10 pixels.

To prevent duplicate alarms within a single maneuver, the system applies a time lockout equivalent to 3 seconds of real time for a given object ID.

5.3. Close-Proximity Overtaking (Code C)

5.3.1. Assumptions

Detection of overtaking or passing maneuvers at and immediately ahead of pedestrian crossings.

5.3.2. Event Analysis

The overtaking zone is initiated dynamically when the nearest radar cluster crosses a boundary distance of 60 meters. Based on the lowest edge of the closest vehicle's bounding box, a virtual division line is determined in pixels. Objects located below this line (closer to the crossing) are analyzed for their mutual spatial arrangement along the depth axis. The Depth-Anything-V2 model is used for this purpose, generating a depth map of the frame. Vehicles are then assigned a relative distance ranking.

5.3.3. Criteria for Raising an Alarm

The system compares position vectors from the previous and current frames. If a unique vehicle ID that was previously further back in the geometric ranking overtakes the ID of the vehicle driving ahead of it, a sequence change is immediately registered, and an overtaking alarm is raised.

6. Input Data

6.1. Initialization Script

During the initial startup, the system performs the following preparatory operations:

- Opens the video file, reads the FPS parameter, and calculates the duration of a single frame. It reads the radar startup delay from the first line of the CSV file.
- Initializes a radar class object, applies a distance mask, injects simulated measurement noise, and launches an automated procedure to search for lane boundaries within the point cloud.
- Executes a weather query for the given latitude and longitude, retrieving current friction coefficients.
- Fetches the first reference frame and generates a baseline scene depth map, removing static vehicles from it.
- Launches the lane detection module to determine straight lines, calculates the road width in pixels, and saves the complete configuration profile to a configuration file.

6.2. Dataset

The testing and validation dataset has been structured into two main groups of traffic scenarios:

- alarm – a directory containing sequences with registered incidents and dangerous situations.
- noalarm – a directory containing control scenarios with situations that should not trigger the system.

Each test case consists of a pair of synchronized files with identical names: a video recording in .mp4 format and radar data in .csv format, located in their respective subfolders.

The file and directory names explicitly encode the types of hazards occurring in a given scenario using unambiguous letter designations. In the event of a cumulative set of dangerous events, the codes are combined (e.g., 10_AC):

- **A** – speed limit exceeded.
- **B** – uncontrolled lane departure.
- **C** – dangerous overtaking maneuver in the crossing zone.

All incoming data was standardized regarding the geometry and mounting constants of the sensor suite. Scenarios were recorded with the camera suspended at a height of 5.5 meters above the roadway surface and at a fixed vertical optical axis inclination angle (Pitch = -6°). Radar data is collected for 700 and 14,000 sample points.

6.3. Input CSV Data Format

Radar data is delivered in a .csv format arranged chronologically by timestamp. The data undergoes geometric correction .

Input columns (raw radar data):

- timestamp - time of point registration from the start of measurement (in seconds), used for synchronization with video frames.
- radial_velocity - radial velocity of the object. Multiplied by -1 in the algorithm so that it reflects the actual approach speed toward the pedestrian crossing.
- x_sensor, y_sensor, z_sensor - raw object coordinates (lateral position, longitudinal distance, and height) in meters relative to the physical sensor.

Columns after geometric transformation:

- x_corrected - lateral position corrected by the automatically calculated horizontal yaw angle (Yaw).
- y_corrected - longitudinal distance of the vehicle to the crossing, corrected by the lateral roll angle (Roll).

- `z_corrected` - real height above the roadway. It results from transforming the raw height and adding a mounting offset (6.0 m), which zeroes the reference system at ground level.

7. Hardware Platform

7.1. Test platform:

- Raspberry Pi 4 Model B
- Google Coral TPU

7.2. Proposed Target Platform:

To eliminate performance bottlenecks of the testing platform and ensure reliability in industrial environments, components for the target hardware architecture were selected without budget constraints.

Component Class	Device Name	Usage in the System
Central Processing Unit	NVIDIA Jetson Orin Nano 8GB	Main edge AI processor, data fusion, and network inference
Radar Sensor	smartmicro TRUGRD LR	Long Range – continuous 3D road scanning and radial velocity measurement
Vision System	AXIS P1488-LE Bullet Camera	4K network camera, image generation for YOLO and depth maps
Traffic Signaling	Dwukomorowy sygnalizator LED Grupy ZIR	Visual cascade alarm (yellow warning light / red alarm light)
Audio Signaling	AXIS C1310-E Mk II Network Horn	Emitting voice/acoustic messages (PoE)
Communication Node	PLANET ISW-514PTF-v4	Industrial network switch with PoE+ and Power Boost support
Executive Module	KAmoDRPi PwrRELAY4	Array of 4 relays with full galvanic optoisolation

Table 2 - List of target platform components.

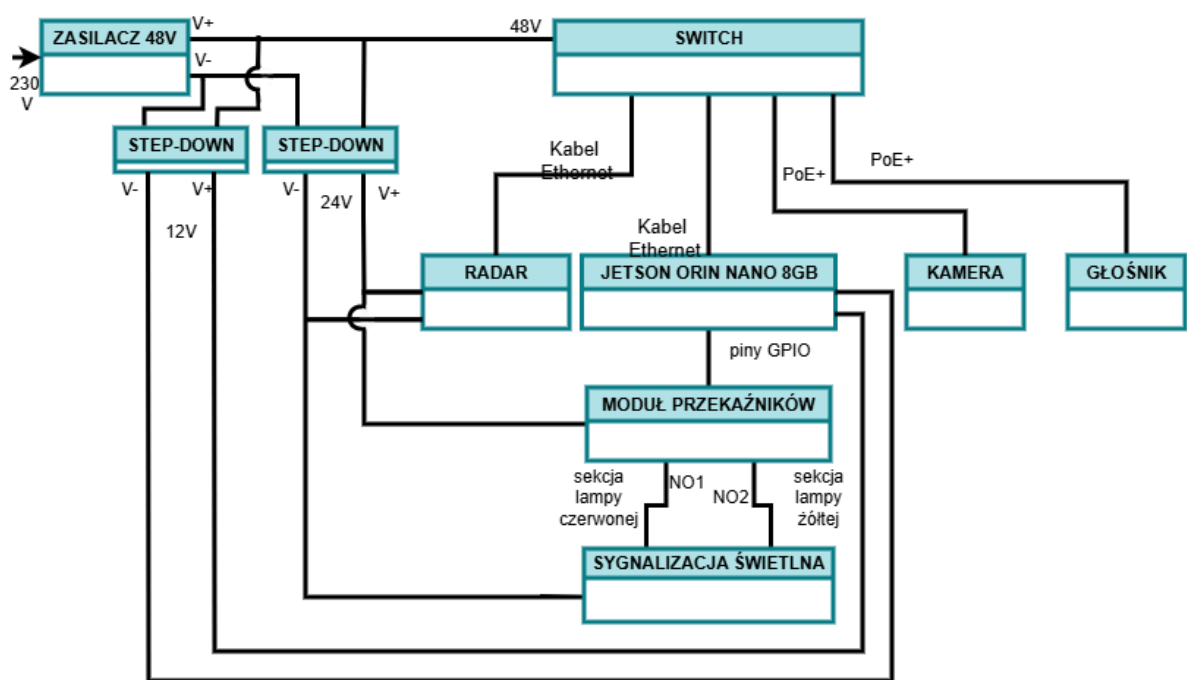


Figure 1 - Block diagram of the target platform.

Technical Description and Role in System Architecture:

The central unit of the system is the NVIDIA Jetson Orin Nano 8GB AI computer. Thanks to its NVIDIA Ampere architecture with 1024 CUDA cores, this module provides a massive computing performance of up to 67 TOPS, allowing parallel processing of a 4K video stream and radar data in real time.

The vision system is based on the professional AXIS P1485-LE network camera. This device is equipped with a 4K sensor and a 10.8–28.2 mm varifocal lens, which allows for precise adjustment of a narrow 14° field of view (FOV). The camera features integrated Lightfinder 2.0 and Forensic WDR technologies, optimizing the image for vehicle detection under extremely difficult lighting and weather conditions.

The TRUGRD LR radar sensor was chosen due to its detection range of 3 to 500 m and high object positioning accuracy. The radar operates continuously, scanning the road and transmitting data packets regarding object speed, distance, and classification directly to the central unit via an Ethernet interface, which guarantees high throughput and transmission stability.

The communication system relies on the industrial PLANET ISW-514PTF-v4 switch. It acts as the central network node, connecting all IP components. The switch is equipped with a Power Boost function, which allows powering the camera and speaker under the PoE+ (802.3at) standard.

The warning system consists of two independent modules:

- Visual: utilizing a two-chamber road LED traffic signal from the ZIR Group in warning yellow and alarm red colors.
- Acoustic: based on the AXIS C1310-E Mk II IP horn speaker.

The KAmoDRPi PwrRELAY4 executive module intermediates the control of the light signal. It acts as a galvanic isolator, effectively separating the GPIO pins of the Jetson system from the 48 V executive circuits. The use of optoisolation protects the central unit from voltage surges and the potential effects of short circuits in the road lamp power supply circuit.

Power for most components is provided by a MEAN WELL NDR-240-48 (5 A, 240 W) switching power supply. It covers the energy demands of the network switch and PoE+ devices. This power supply is also the current source for the radar and traffic signals, whose operating voltage is lowered to 24 V DC using a dedicated step-down converter. The Jetson Orin Nano unit is powered with 12V thanks to a step-down converter.

8. Setup Guide

8.1. Environment Installation

Hardware Requirements:

- Raspberry Pi 4 B device (8 GB RAM version recommended to prevent I/O delays caused by memory paging).
- Google Coral USB Accelerator connected to the blue USB 3.0 port.

Automated Installation:

The deployment has been fully integrated within the setup.sh script. The script independently verifies and, if necessary, downloads the Miniforge3 (Conda)

environment, installs system libraries, configures the high-performance driver for the Coral co-processor, and sets the CPU performance profile.

8.2. Test Platform Configuration

The installation script autonomously carries out the following configuration operations within the Raspberry Pi system environment:

- **Verification and Installation of the Conda Environment (Miniforge3):** The script checks for the presence of the conda tool in the system. If missing, it automatically downloads the official Miniforge3 distribution installer dedicated to the ARM64 (aarch64) architecture. It then deploys the software in the user's home directory and initializes the Bash shell, guaranteeing the availability of the Conda environment during every new terminal session.
 - **System Package Update:** The apt-get update and apt-get upgrade commands are initiated to synchronize repositories and install the latest security patches for the Debian Bookworm system.
 - **Installation of the High-Performance Google Coral Driver:** The system verifies the presence of the standard, performance-limited libedgetpu1-std (250 MHz) driver and, if detected, removes it along with its configuration files. In the next step, the optimized version libedgetpu1-max, compatible with Debian Bookworm and the ARM64 architecture, is downloaded and installed. This action unlocks the maximum clock frequency of the Coral co-processor (500 MHz), which only requires disconnecting and reconnecting it to the USB port after installation is complete.
 - **CPU Performance Optimization (CPU Governor):** The default core clock frequency governor (usually ondemand or schedutil) is modified to the performance profile. This locks the clock frequency of all CPU cores at the maximum level of 1.5 GHz, effectively eliminating delays and performance fluctuations resulting from sudden spikes in load. To maintain this setting after every reboot, the script appends the appropriate directive to the /etc/rc.local configuration file.
 - **Deployment of the Conda Virtual Development Environment:** The script checks whether a virtual environment named zsd is already operational. If not, it is automatically generated based on the dependency definitions in the environment.yaml file. In the final phase, the script activates the newly created environment and updates the ultralytics framework to the latest available version.
-

9. Achieved Results

The developed system fully realized the original project assumptions and significantly exceeded them in key metrics of efficiency and fault tolerance.

- Safety and Stability (FAR): The system demonstrated complete resistance to raising unjustified alarms. The margin of error allowed in the original project assumptions was entirely eliminated, confirming the high stability of the algorithms and a lack of susceptibility to generating false alarms.
- Detection Effectiveness (TAR): The developed algorithms exceeded the required minimum effectiveness threshold, flawlessly identifying real threat states on the road.
- The system achieved a very high scenario classification accuracy of 88% across the entire database of diverse validation recordings.
- Full Implementation of functionality and sensor fusion
- Threat Detection (Codes A, B, C): The system architecture successfully integrates verification mechanisms for speeding linked to braking distance estimation (A), precise monitoring of lane line crossings while maintaining a strict technical tolerance (B), and relative longitudinal positioning of objects during overtaking maneuvers (C).
- Geometric and Hardware Adaptation: The detection logic was optimally tuned to the target mounting geometry of the sensor suite. Utilizing the advanced DBSCAN spatial segmentation algorithm allowed for stable detection of approaching vehicle clusters and isolated them from infrastructure elements.
- Dynamic Weather Adaptation: Fulfilling the assumption of working in variable conditions, integration with the external Open-Meteo API was introduced into the system. The system automatically retrieves weather codes and adjusts the physics of the braking distance and driver reaction time for dry, rainy, and slippery (black ice) surfaces.

The system's effectiveness was verified in a batch test (Batch Evaluation) on a sample of 60 test recordings representing diverse road scenarios.

Output Metrics :

- Quantity of recordings : 60
- Correct classifications : 53
- Accuracy : 88%
- False alarm rate (FAR) : 13%
- True alarm rate (TAR) : 89%

18 safe-scene recordings: The system achieved very high effectiveness (FAR = 13%). In 16 out of 18 trials, the no-threat status was correctly assigned despite the presence of significant environmental noise (up to 16 non-threatening objects

detected and classified as other in the background of the scene). The DBSCAN filtering and point cloud masking methods effectively isolated neutral objects.

42 hazardous-scene recordings: The system correctly recognized the threat condition and the complete set of required alarm codes in 37 out of 42 recordings (TAR $\approx$ 89%). In 5 cases, discrepancies were observed in the structure of the recognized scenario codes.
